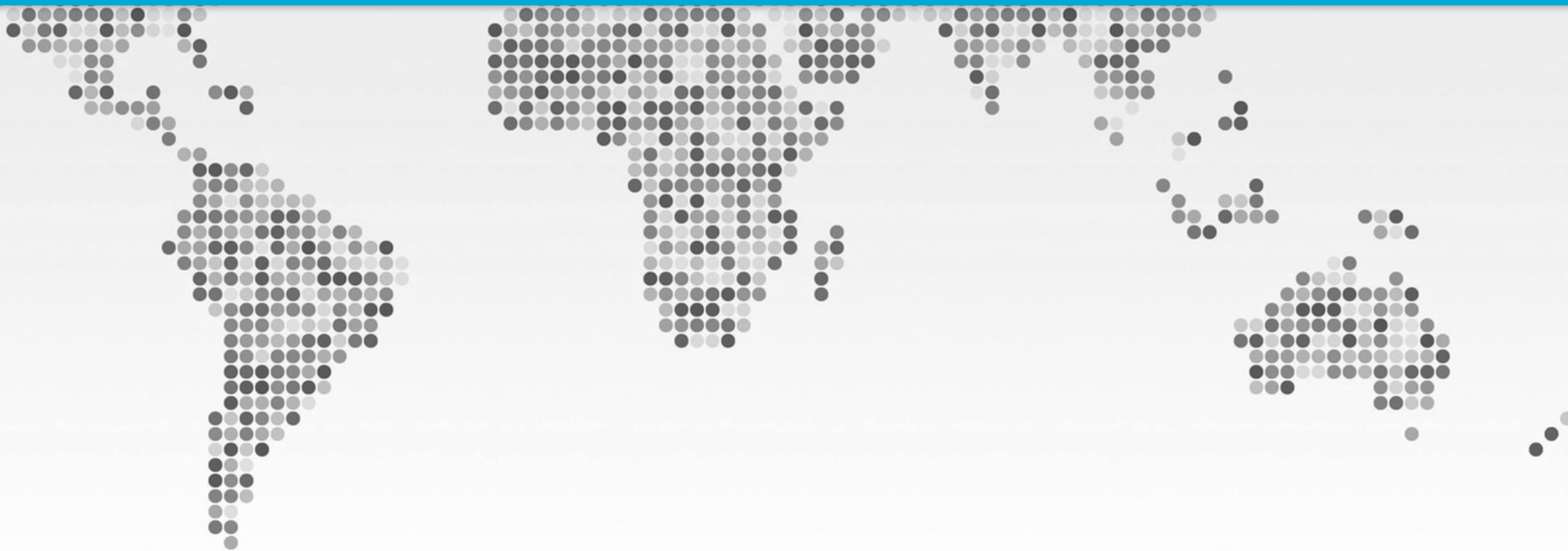


Maven

Laboratorio di Ingegneria del Software



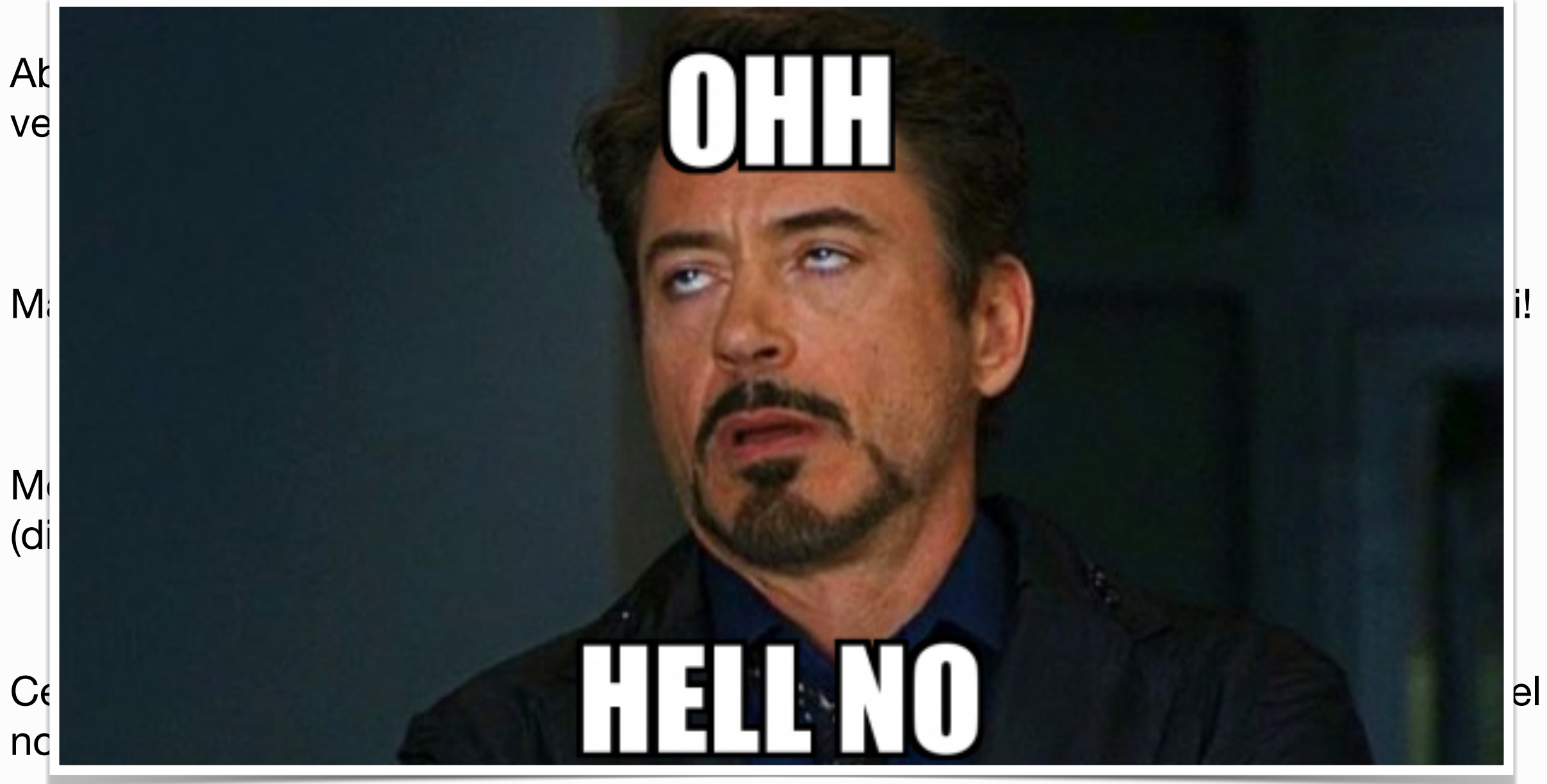
Abbiamo visto che Git fornisce la possibilità di tenere sotto controllo le diverse versioni dei nostri progetti e di collaborare con i nostri colleghi.

Ma i nostri progetti non sono costituiti esclusivamente da codice che scriviamo noi!

Molto spesso utilizziamo codice scritto da qualcun altro, importando le librerie (dipendenze del progetto).

Certo, potremo facilmente includere manualmente le classi Java di qualcun altro nel nostro progetto.

Tuttavia, dovremmo gestire questa operazione manualmente e capire come integrare le nostre componenti. E se queste nuove componenti vengono aggiornate **dovremo ripartire da zero.**



Tuttavia, dovremmo gestire questa operazione manualmente e capire come integrare le nostre componenti. E se queste nuove componenti vengono aggiornate **dobbiamo ripartire da zero.**

Inoltre, immaginiamo di importare una **dipendenza manualmente** nel nostro progetto, ad esempio **aggiungendo un JAR** alla cartella, possiamo poi tenere sotto controllo tutto attraverso Git. Giusto?

Gestione delle Dipendenze

Inoltre, immaginiamo di importare una **dipendenza manualmente** nel nostro progetto, ad esempio **aggiungendo un JAR** alla cartella, possiamo poi tenere sotto controllo tutto attraverso Git. Giusto?



Inoltre, immaginiamo di importare una **dipendenza manualmente** nel nostro progetto, ad esempio **aggiungendo un JAR** alla cartella, possiamo poi tenere sotto controllo tutto attraverso Git. Giusto?

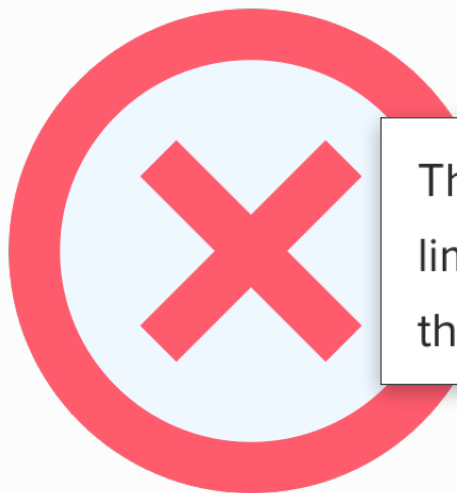


**Failed to resolve
dependencies when
building the application**

Inoltre, immaginiamo di importare una **dipendenza manualmente** nel nostro progetto, ad esempio **aggiungendo un JAR** alla cartella, possiamo poi tenere sotto controllo tutto attraverso Git. Giusto?



Failed to resolve dependencies when building the application



The push was rejected because the repository would have exceeded the 10.0 GB limit for Git repository in Backlog. Please reduce the size of the files included in the commits and push again.

Inoltre, immaginiamo di importare una **dipendenza manualmente** nel nostro progetto, ad esempio **aggiungendo un JAR** alla cartella, possiamo poi tenere sotto controllo tutto attraverso Git. Giusto?



Failed to resolve
dependencies when

Abbiamo bisogno di un tool che gestisce le dipendenze con componenti esterne in modo **totalmente separato.**

limit for Git repository in Backlog. Please reduce the size of the files included in the commits and push again.



Maven è un tool di build che mette a disposizione la **gestione automatica** delle dipendenze.



Maven è un tool di build che mette a disposizione la **gestione automatica** delle dipendenze.

In parole più semplici, noi diciamo quali librerie abbiamo bisogno per poter effettuare la build del nostro progetto, e Maven le recupera per noi.



Maven è un tool di build che mette a disposizione la **gestione automatica** delle dipendenze.

In parole più semplici, noi diciamo quali librerie abbiamo bisogno per poter effettuare la build del nostro progetto, e Maven le recupera per noi.

Inoltre, Maven si occuperà di risolvere le **dipendenze transitive**. Ad esempio: se necessitiamo di una libreria A, che a sua volta dipende da una libreria B, Maven importerà **anche** la libreria B.

Come possiamo “dire” a Maven le dipendenze di cui abbiamo bisogno?

Attraverso il Project Object Model (POM).

Il Project Object Model è un file di configurazione XML che descrive:

- il progetto
- i suoi dettagli (come la versione Java da utilizzare)
- e le sue dipendenze.

Nel file pom.xml codifichiamo il Project Object Model.

Il file pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>it.unisa</groupId>
  <artifactId>progetto-is</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>19</maven.compiler.source>
    <maven.compiler.target>19</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.9.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

</project>
```

Il nostro progetto

Il file pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>it.unisa</groupId>
  <artifactId>progetto-is</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>19</maven.compiler.source>
    <maven.compiler.target>19</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.9.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

</project>
```

I dettagli

Il file pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>it.unisa</groupId>
  <artifactId>progetto-is</artifactId>
  <version>1.0-SNAPSHOT</version>

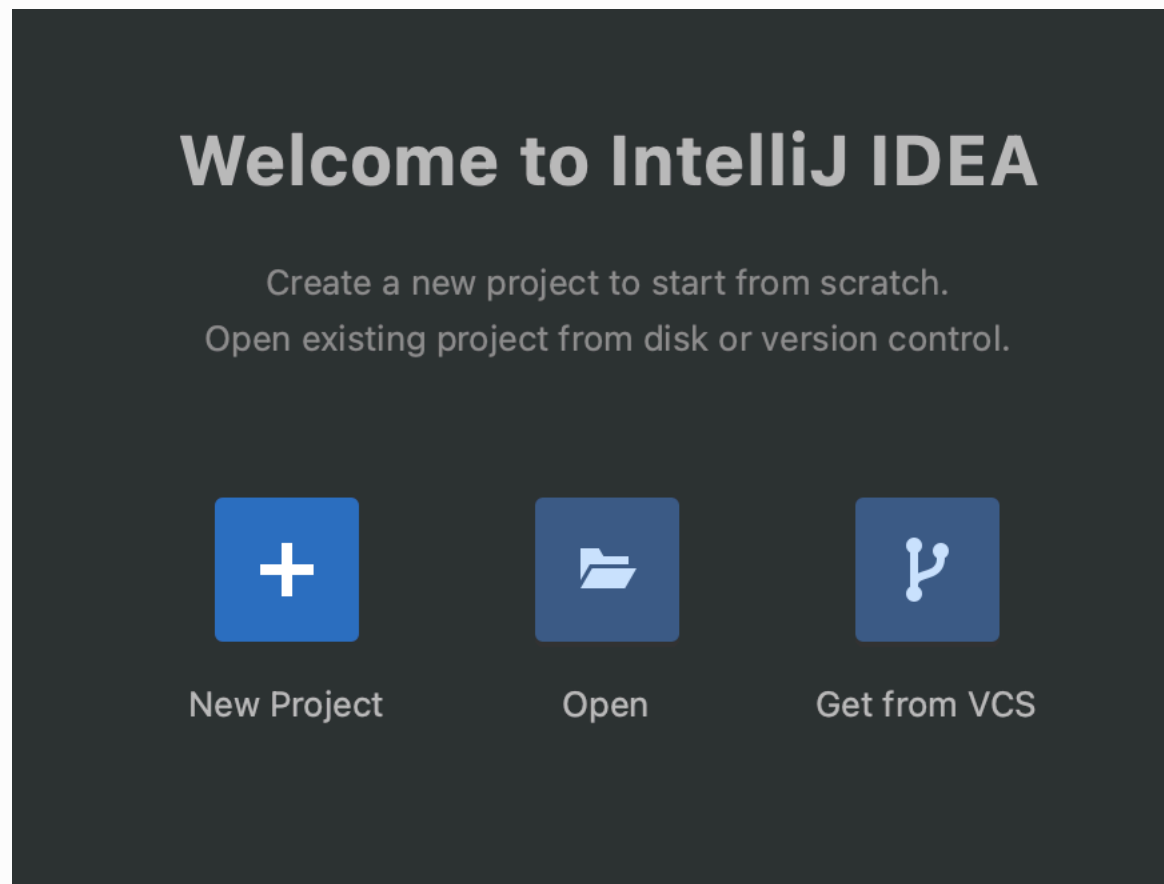
  <properties>
    <maven.compiler.source>19</maven.compiler.source>
    <maven.compiler.target>19</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.9.0</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

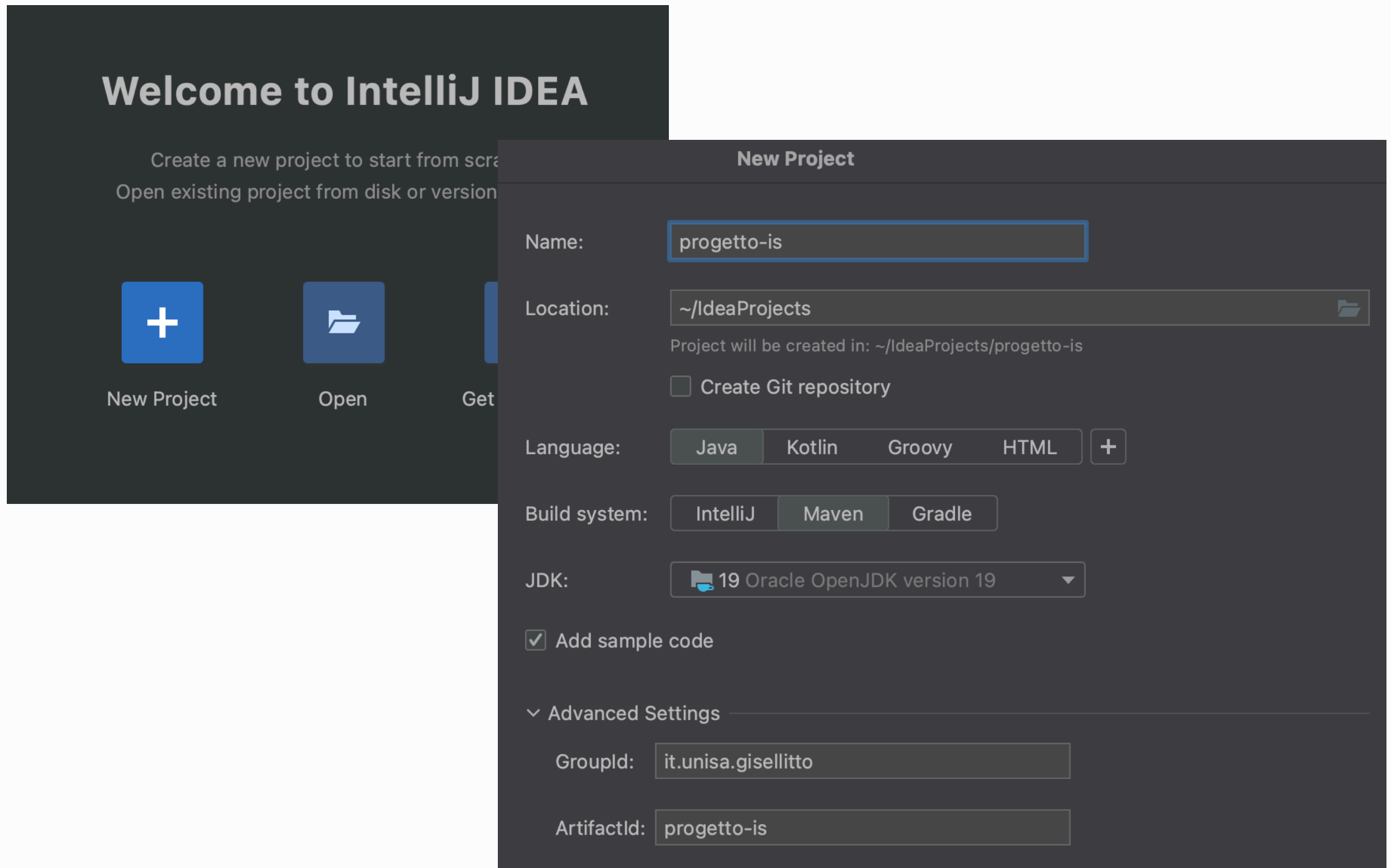
</project>
```

Le dipendenze

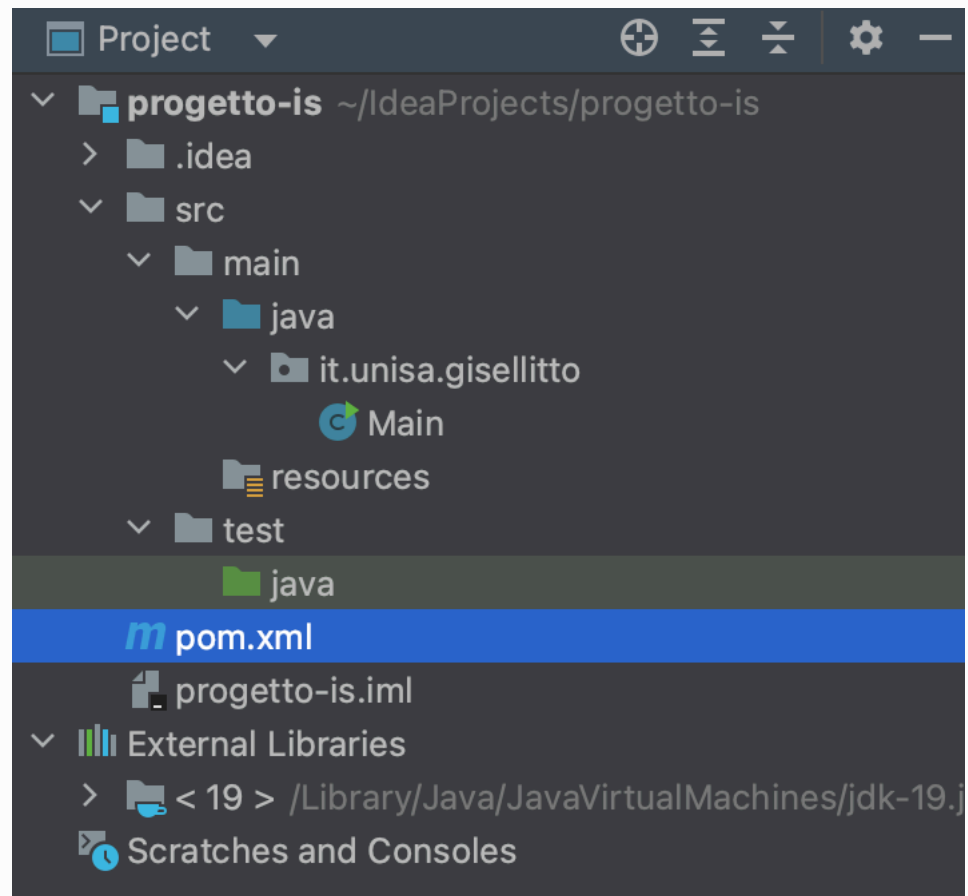
Creiamo un progetto Maven in IntelliJ.



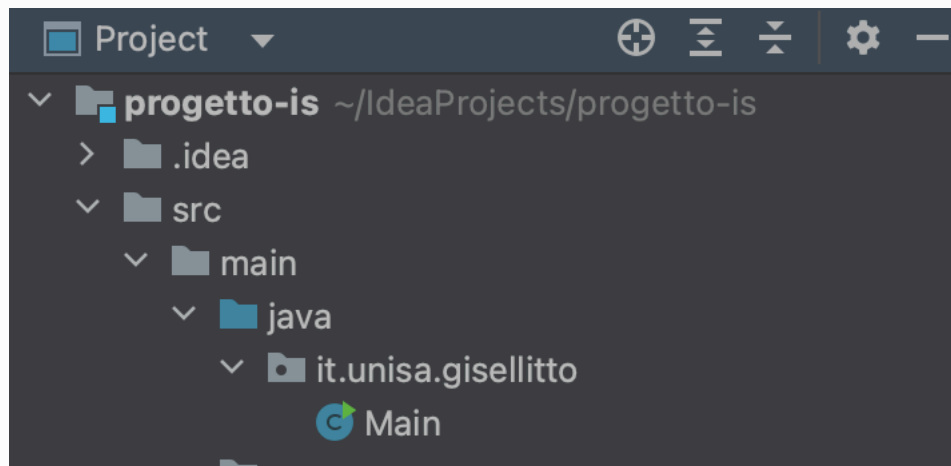
Creiamo un progetto Maven in IntelliJ.



Struttura di un progetto Maven



Struttura di un progetto Maven



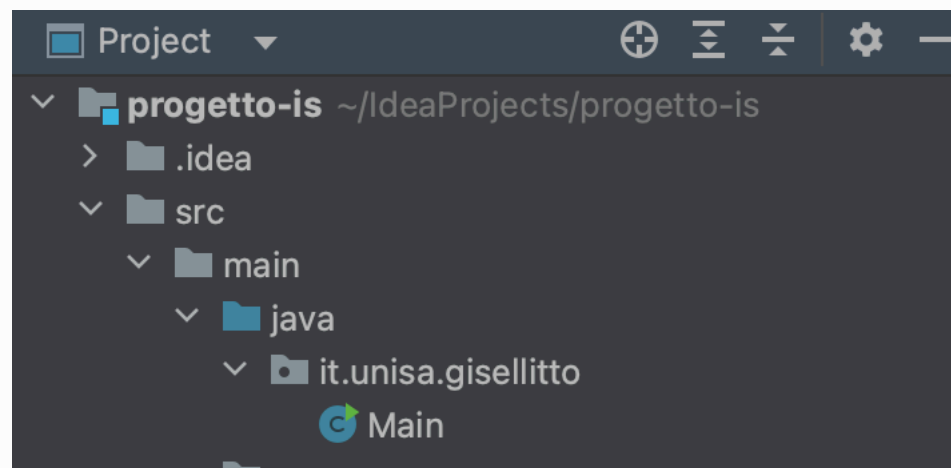
```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>it.unisa.gisellitto</groupId>
  <artifactId>progetto-is</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>19</maven.compiler.source>
    <maven.compiler.target>19</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

</project>
```

Struttura di un progetto Maven



```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>it.unisa.gisellitto</groupId>
  <artifactId>progetto-is</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>19</maven.compiler.source>
    <maven.compiler.target>19</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

</project>
```

Ci mancano le
dipendenze

Importiamo una dipendenza dalla Maven Repository



Repository

- Central 2.3k
- Sonatype 672
- Spring Plugins 344
- Spring Lib M 305
- JCenter 124
- IBiblio 71
- Spring Lib Release 66
- Geomajas 55

Group

- com.github 294
- io.github 89
- org.apache 88
- org.eclipse 82
- com.javiersc 48
- org.junit 45
- org.powermock 42
- org.testifyproject 33

Found 2751 results

Sort: **relevance** | popular | newest

- 

1. JUnit Jupiter API
[org.junit.jupiter » junit-jupiter-api](#)
 JUnit Jupiter is the API for writing tests using JUnit 5.
 Last Release on Sep 20, 2022
11,304 usages
[EPL](#)
- 

2. JUnit
[junit » junit](#)
 JUnit is a unit testing framework to write and run repeatable automated tests on Java.
 Last Release on Feb 13, 2021
119,318 usages
[EPL](#)
- 

3. JUnit Jupiter Engine
[org.junit.jupiter » junit-jupiter-engine](#)
 Core package for the JUnit Jupiter test engine.
 Last Release on Sep 20, 2022
11,093 usages
[EPL](#)

<https://mvnrepository.com/>

Importiamo una dipendenza dalla Maven Repository

[Home](#) » [org.junit.jupiter](#) » junit-jupiter-api



JUnit Jupiter API

JUnit Jupiter is the API for writing tests using JUnit 5.

License	EPL 2.0
Categories	Testing Frameworks
Tags	junit testing api
Ranking	#44 in MvnRepository (See Top Artifacts) #3 in Testing Frameworks
Used By	11,304 artifacts

Central (58)

ICM (2)

	Version	Vulnerabilities	Repository	Usages	Date
5.9.x	5.9.1		Central	33	Sep 20, 2022
	5.9.0		Central	860	Jul 26, 2022
	5.9.0-RC1		Central	79	Jul 04, 2022
	5.9.0-M1		Central	106	May 15, 2022
	5.8.2		Central	2,247	Nov 28, 2021
	5.8.1		Central	1,101	Sep 22, 2021

<https://mvnrepository.com/>

Importiamo una dipendenza dalla Maven Repository

[Home](#) » [org.junit.jupiter](#) » junit-jupiter-api



JUnit Jupiter API

JUnit Jupiter is the API for writing tests using JUnit 5.

License	EPL 2.0
Categories	Testing Frameworks
Tags	junit testing api
Ranking	#44 in MvnRepository (See Top Artifacts) #3 in Testing Frameworks
Used By	11,304 artifacts

Central (58) ICM (2)

	Version	Vulnerabilities	Repository	Usages	Date
5.9.x	5.9.1		Central	33	Sep 20, 2022
	5.9.0		Central	860	Jul 26, 2022
	5.9.0-RC1		Central	79	Jul 04, 2022
	5.9.0-M1		Central	106	May 15, 2022
	5.8.2		Central	2,247	Nov 28, 2021
	5.8.1		Central	1,101	Sep 22, 2021

<https://mvnrepository.com/>

Importiamo una dipendenza dalla Maven Repository

[Home](#) » [org.junit.jupiter](#) » junit-jupiter-api



JUnit Jupiter API

JUnit Jupiter is the API for writing tests using JUnit 5.

License	EPL 2.0
Categories	Testing Frameworks
Tags	junit testing api
Ranking	#44 in MvnRepository (See Top Artifacts) #3 in Testing Frameworks
Used By	11,304 artifacts

Maven

Gradle

Gradle (Short)

Gradle (Kotlin)

SBT

Ivy

Grape

Leiningen

Buildr

```
<!-- https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-api -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.8.2</version>
  <scope>test</scope>
</dependency>
```

☒ Include comment with link to declaration

<https://mvnrepository.com/>

Importiamo una dipendenza dalla Maven Repository

[Home](#) » [org.junit.jupiter](#) » junit-jupiter-api



JUnit Jupiter API

JUnit Jupiter is the API for writing tests using JUnit 5.

License	EPL 2.0
Categories	Testing Frameworks
Tags	junit testing api
Ranking	#44 in MvnRepository (See Top Artifacts) #3 in Testing Frameworks
Used By	11,304 artifacts

Maven

Gradle

Gradle (Short)

Gradle (Kotlin)

SBT

Ivy

Grape

Leiningen

Buildr

```

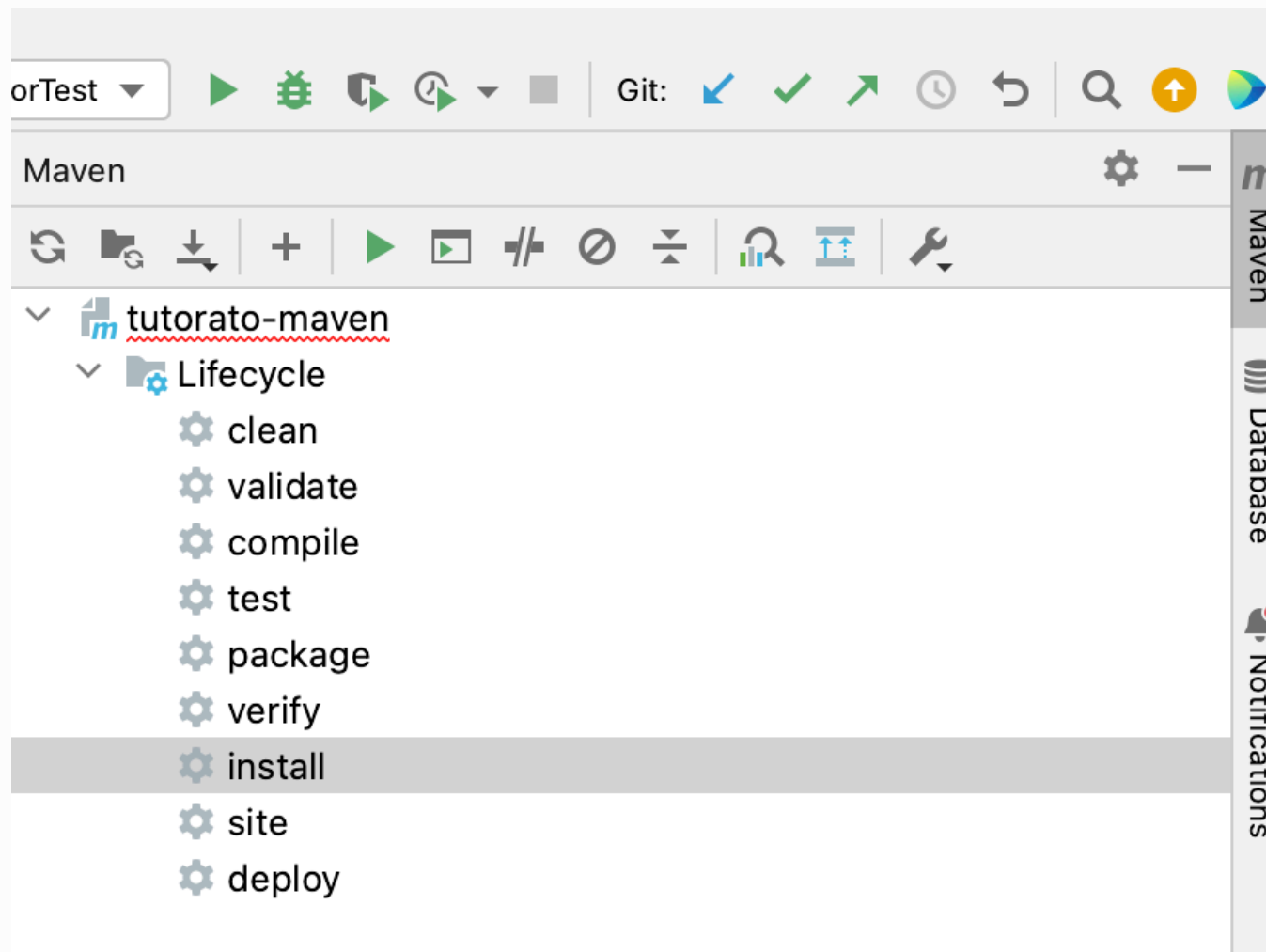
<!-- https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-api -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.8.2</version>
  <scope>test</scope>
</dependency>
    
```

E questo?

☒ Include comment with link to declaration

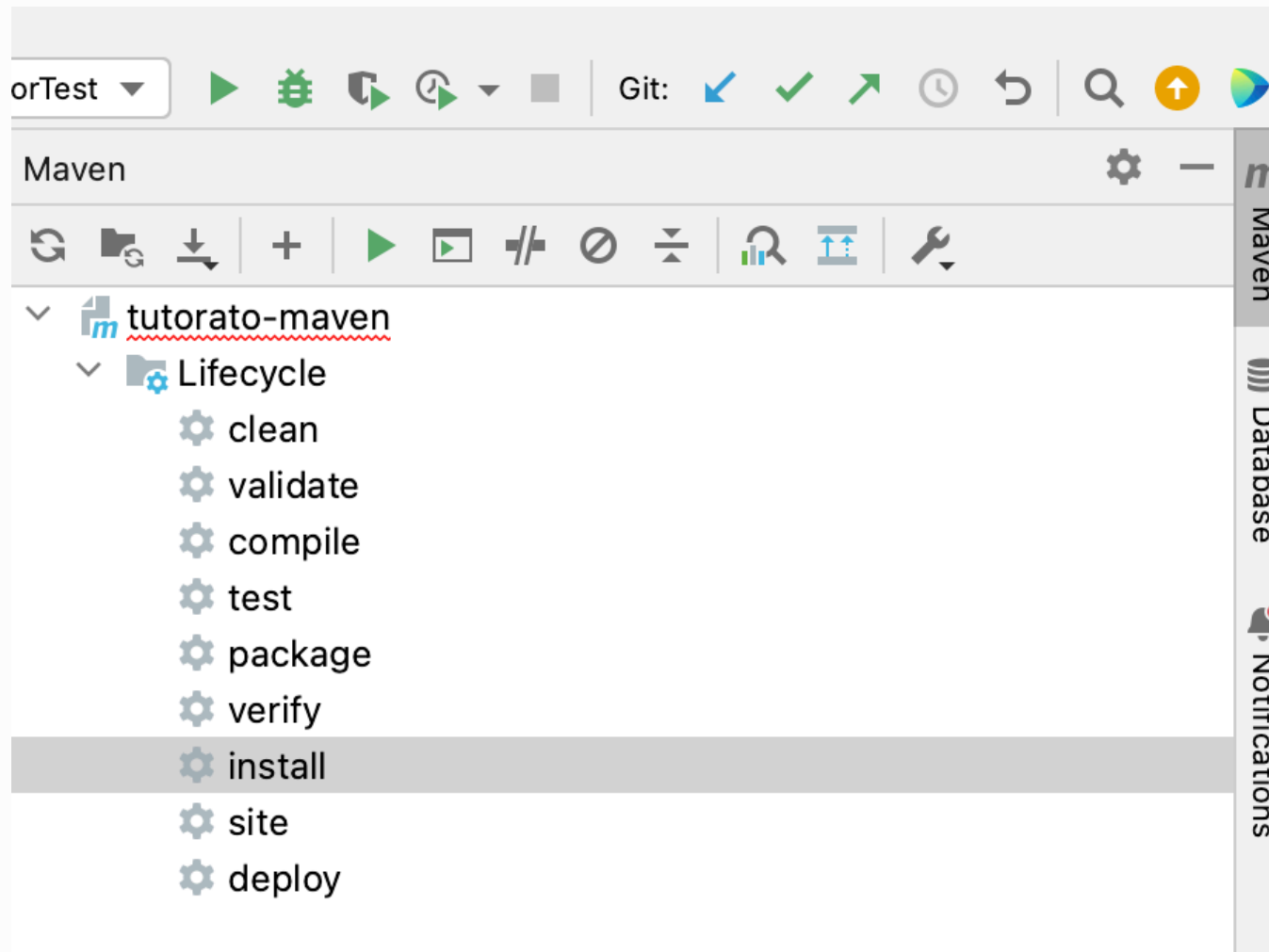
<https://mvnrepository.com/>

I cicli di vita Maven:



<https://mvnrepository.com/>

I cicli di vita Maven:



Consiglio: Prima di ogni run, eseguire sempre il comando **clean** e poi il comando **install**.

<https://mvnrepository.com/>

Maven

Laboratorio di Ingegneria del Software

